

图 7.9: HID Boot 键盘报告（8 字节）。byte 0 是 8 个修饰键的位图，byte 2 ~ 7 是当前按着的最多 6 个键的 usage 编号

platform 设备和驱动；因此 `dwc2_probe()` 发现并注册 USB 设备时，`device_register()` 可以立即找到已经存在的 `usbkbd_driver` 并完成绑定。

调用链中有三个容易混淆的边界：

1. **USB 设备地址与 DWC2 channel 不是同一个编号**。`SET_ADDRESS(4)` 改变的是总线上的设备地址；ch0、ch6、ch7 是 DWC2 内部的事务执行上下文。`dwc2_submit_urb()` 把 URB 中的 device address、endpoint、方向、速度和 split route 写入 ch6 寄存器，才建立“ch6 正在服务 addr 4 的 EP1-IN”这一临时关系。
2. **枚举和运行采用不同执行模型**。`control()` 在启动阶段同步等待 ch0 完成，便于严格保持 USB 枚举顺序；键盘运行期则由 ch6 IRQ 完成 URB，再由 workqueue 解析 HID report，NAK 重试通过 `dwc2_periodic_wq` 的 delayed work 按到期时间重提。
3. **注册设备不是重新枚举**。`usb_device_register()` 只把已完成地址和配置阶段的 `usb_child` 交给设备模型；`usb_match()` 选择驱动，`usbkbd_probe()` 完成 HID 类请求、输入设备注册和第一只运行期 URB 的提交。